

ACTIVIDAD AUTÓNOMA N.º 08

Pruebas de operaciones CRUD y errores controlados

Estudiante	Juan Paredes Cárdenas
Curso	Desarrollo de Aplicaciones Móviles
Docente	Benjamín David Reyna Barreto
Proyecto	PharmaMobil - Kotlin Multiplatform
Fecha	1 de octubre de 2026

Rama de entrega:
github.com/DJuanParedes/pharmaMobil/tree/actividad-autonoma-sesion-8

1. Resumen de la actividad

Se amplió el módulo de Productos para comprobar, de forma repetible, las operaciones CRUD REST y el tratamiento de fallos antes de que lleguen a la interfaz. La aplicación traduce respuestas HTTP, problemas de conexión, tiempo de espera y cancelaciones a estados de dominio comprensibles. Las evidencias se obtuvieron en un emulador Pixel 6 API 37 y mediante pruebas de commonTest.

Resultado general: el proyecto compila, el APK se genera y las 54 pruebas del módulo compartido finalizan sin fallos ni errores.

Criterios aplicados

Aspecto	Aplicación en PharmaMobil
Contrato REST	GET listado, GET por id, POST, PUT y DELETE con Ktor Client.
Errores controlados	ErrorApi.Validacion, NoEncontrado, Conflicto, Servidor, SinConexion, TiempoAgotado y RespuestaInvalida.
Estado de UI	Cargando, ConProductos, SinProductos y Error; operación Inactiva, EnCurso o Fallida.
Verificación	Mock REST local para el recorrido Android y MockEngine para pruebas aisladas.

La API local utilizada para las evidencias reproduce los códigos y cuerpos esperados del backend. No reemplaza la validación de integración con un servidor institucional desplegado, pero permite comprobar el comportamiento del cliente de forma controlada y documentada.

Limitación de plataforma: esta ejecución se realizó en Windows. El código conserva el objetivo iOS y su URL local, pero una captura real de iPhone requiere macOS y Xcode. Por integridad académica, este informe no presenta una evidencia iOS simulada.

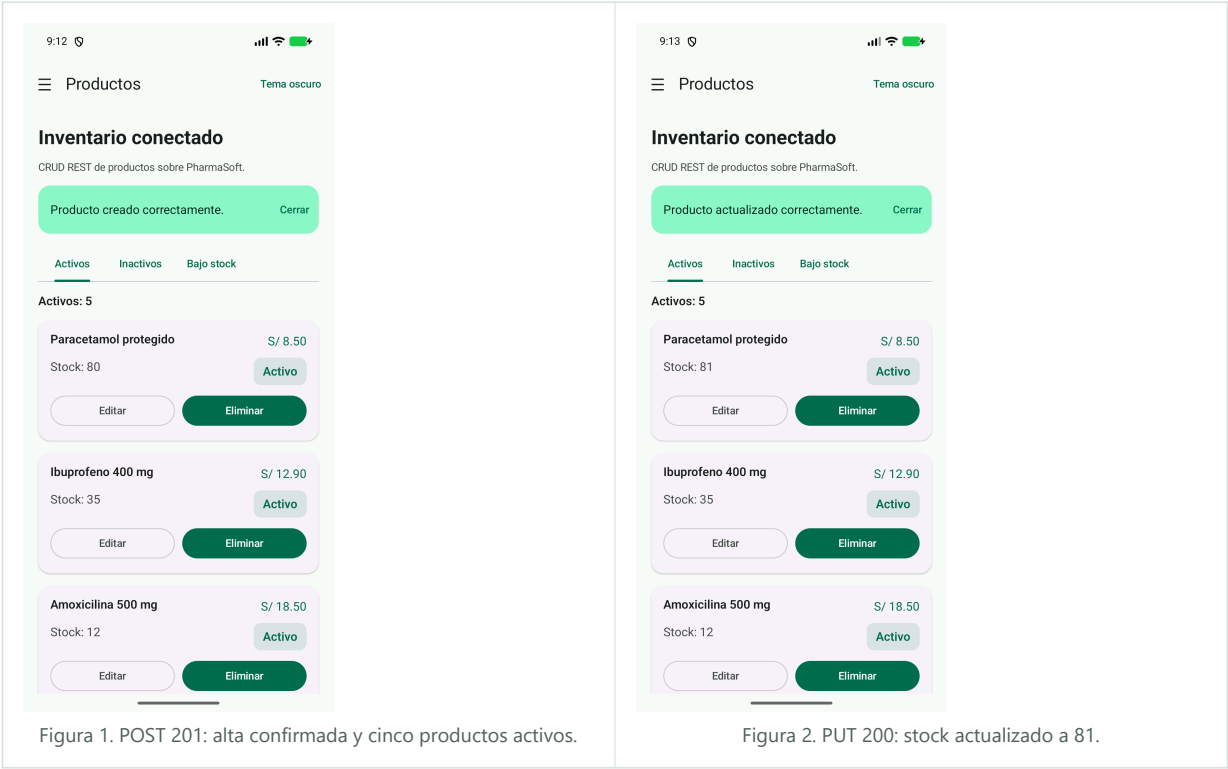
2. Matriz de operaciones CRUD

Las cinco operaciones fueron comprobadas contra el servicio local y también están cubiertas por ProductoApiTest con MockEngine.

Operación	Solicitud	Estado	Resultado observado
Listar	GET /api/v1/productos?pagina=0&tamano=20&ordenarPor=nombre&direccion=asc	200 OK	Se recibió una página con cuatro productos activos.
Consultar	GET /api/v1/productos/1	200 OK	Se obtuvo Paracetamol protegido con precio S/ 8.50, stock 80 y estado activo.
Crear	POST /api/v1/productos {nombre: Vitamina C, precio: 12.50, stock: 20, estado: true, categoriald: 1}	201 Created	Vitamina C fue creada; la lista pasó de cuatro a cinco productos activos.
Actualizar	PUT /api/v1/productos/1 {nombre: Paracetamol protegido, precio: 8.50, stock: 81, estado: true, categoriald: 1}	200 OK	El stock cambió de 80 a 81 y el listado se recargó.
Eliminar	DELETE /api/v1/productos/1000	204 No Content	Vitamina C desapareció y la lista volvió a cuatro productos activos.

Lectura de la matriz

La recarga posterior a POST, PUT y DELETE evita que la UI quede con datos antiguos. El registro de Logcat confirma los códigos 201, 200 y 204; la captura del listado comprueba el efecto visible de cada operación.



3. Bitácora de errores controlados

Cada caso registra el recorrido realizado, el tipo de error esperado y la respuesta visible para el usuario.

Caso 1 - Validación del servidor	
Fecha, hora y plataforma	29/09/2026, 21:10 - Android, Pixel 6 API 37
Pasos	Abrir Productos; registrar "Paracetamol protegido" con datos válidos; enviar el formulario.
HTTP / ErrorApi	400 Bad Request / ErrorApi.Validacion(nombre)
Mensaje al usuario	Ya existe un producto con este nombre.
Evidencia	04-validacion-servidor.png
Conclusión	El formulario conserva los datos y coloca el mensaje debajo de Nombre; la pantalla no cambia a un error general.

Caso 2 - Precio inválido	
Fecha, hora y plataforma	29/09/2026, 21:09 - Android, Pixel 6 API 37
Pasos	Ingresar nombre válido, precio 0 y stock positivo; presionar Crear producto.
HTTP / ErrorApi	No enviado / ValidacionProductoException / campo precio
Mensaje al usuario	El precio debe ser mayor o igual a 0.01.
Evidencia	03-precio-invalido.png
Conclusión	La regla local detiene una petición innecesaria. El caso se resuelve antes de Ktor y mantiene el foco del formulario.

3. Bitácora de errores controlados (continuación)

Caso 3 - Recurso inexistente	
Fecha, hora y plataforma	29/09/2026, 21:11 - Android, Pixel 6 API 37
Pasos	Intentar eliminar el producto desactualizado con id 999; confirmar la acción.
HTTP / ErrorApi	404 Not Found / ErrorApi.NoEncontrado
Mensaje al usuario	El producto ya no existe. Actualiza la lista.
Evidencia	06-recurso-inexistente.png
Conclusión	La app mantiene el inventario visible y ofrece un mensaje accionable; el registro Ktor conserva el 404.

Caso 4 - Doble eliminación	
Fecha, hora y plataforma	29/09/2026, 21:16 - servicio REST local
Pasos	Enviar DELETE al id 2; repetir exactamente la misma solicitud después del 204.
HTTP / ErrorApi	204 y luego 404 / Éxito y después ErrorApi.NoEncontrado
Mensaje al usuario	Producto no encontrado en el segundo intento.
Evidencia	14-doble-eliminacion.png
Conclusión	El endpoint es idempotente desde el estado final: no reaparece el producto y el segundo intento se controla.

3. Bitácora de errores controlados (continuación)

Caso 5 - Conflicto de negocio	
Fecha, hora y plataforma	29/09/2026, 21:11 - Android, Pixel 6 API 37
Pasos	Seleccionar Paracetamol protegido; presionar Eliminar y confirmar.
HTTP / ErrorApi	409 Conflict / ErrorApi.Conflicto
Mensaje al usuario	No se puede eliminar: el producto tiene ventas asociadas.
Evidencia	05-conflicto-negocio.png
Conclusión	Se respeta la regla del backend; el producto permanece en la lista y se muestra el motivo exacto.

Caso 6 - Servidor caído	
Fecha, hora y plataforma	29/09/2026, 21:15 - Android, Pixel 6 API 37
Pasos	Detener el servicio REST; abrir Productos para forzar la carga del inventario.
HTTP / ErrorApi	Sin respuesta HTTP / ErrorApi.SinConexion
Mensaje al usuario	No se pudo conectar con PharmaSoft. Verifica la red y el backend.
Evidencia	11-servidor-caido.png
Conclusión	La app no se cierra, informa el problema y mantiene disponible el botón Reintentar.

Caso 7 - Tiempo de espera	
Fecha, hora y plataforma	29/09/2026, 21:15 - Android, Pixel 6 API 37
Pasos	Configurar una respuesta de 20 s; iniciar la carga con timeout del cliente en 15 s.
HTTP / ErrorApi	Solicitud interrumpida / ErrorApi.TiempoAgotado
Mensaje al usuario	La operación tardó demasiado. Intenta nuevamente.
Evidencia	10-timeout.png
Conclusión	El retraso se diferencia de una desconexión y permite repetir la solicitud.

Caso 8 - Cancelación	
Fecha, hora y plataforma	29/09/2026, 21:00 - commonTest / Android host
Pasos	Iniciar una carga demorada; cancelar el ViewModel antes de que el repositorio responda.
HTTP / ErrorApi	No corresponde / CancellationException propagada
Mensaje al usuario	No se genera mensaje de error falso.
Evidencia	12-pruebas-commonTest.png
Conclusión	La cancelación conserva la semántica de corrutinas y no se transforma en ErrorApi ni ProductoOperacion.Fallida.

4. Evidencias visuales de validación y respuestas HTTP

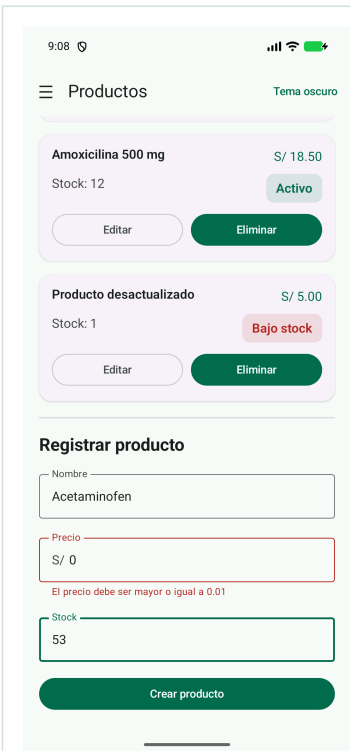


Figura 3. Validación local del precio; la petición no se envía.



Figura 4. HTTP 400 mostrado debajo del campo Nombre.

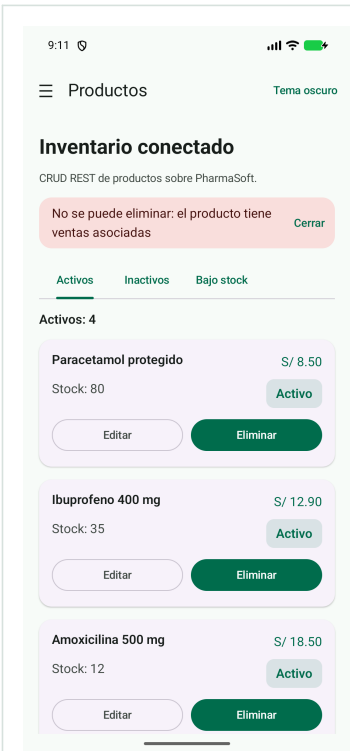


Figura 5. HTTP 409 con el motivo de negocio conservado.



Figura 6. HTTP 404 traducido a un mensaje comprensible.

5. Evidencias de conectividad y recuperación

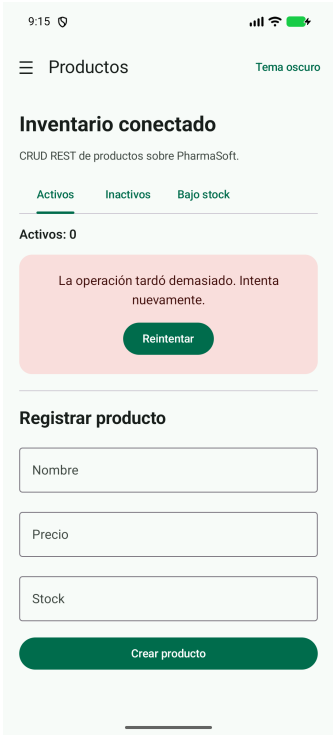


Figura 7. Tiempo de espera: mensaje específico y opción Reintentar.

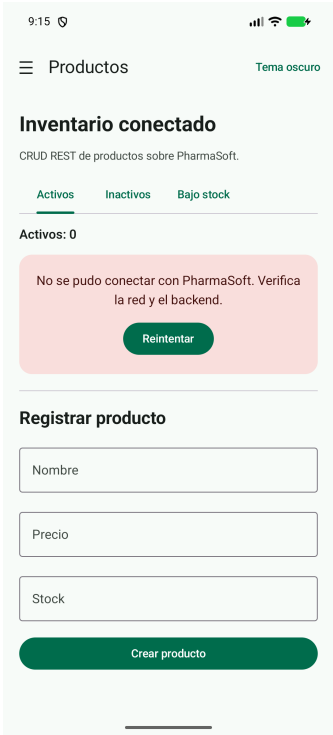


Figura 8. Sin conexión: la aplicación permanece estable.



Figura 9. DELETE 204: eliminación completada y recarga.



Figura 10. Inventario inicial consumido desde el servicio REST.

6. Pruebas de estados y manejo de errores

Se añadieron pruebas en commonTest con un repositorio falso configurable y MockEngine. Esto permite evaluar el ViewModel sin depender de la red y comprobar la traducción de fallos de Ktor.

Prueba	Transición o regla comprobada
Carga exitosa	Cargando → ConProductos.
Listado vacío	Cargando → SinProductos.
Validación 400	El mensaje queda debajo del campo y la fase del listado no se pierde.
Eliminación	Inactiva → EnCurso(Eliminar) → Inactiva, seguida de una nueva carga.
404 y 409	Las respuestas se convierten en NoEncontrado y Conflicto con mensaje.
Red y timeout	IOException → SinConexion; HttpRequestTimeoutException → TiempoAgotado.
Cancelación	CancellationException se propaga y no se convierte en un fallo de UI.

```
PharmaMobil – pruebas automatizadas

> .\gradlew.bat :shared:testAndroidHostTest --console=plain

ProductoViewModelEstadosTest
  PASS carga: Cargando -> ConProductos
  PASS lista vacía: Cargando -> SinProductos
  PASS validación 400 bajo el campo correspondiente
  PASS eliminar: EnCurso -> Inactiva + recarga
  PASS cancelación no se convierte en error de UI

ProductoApiTest
  PASS 404, 409, sin conexión, timeout y cancelación

54 pruebas | 0 fallos | 0 errores | 0 omitidas
BUILD SUCCESSFUL
```

Figura 11. Resumen obtenido de los XML de testAndroidHostTest: 54 pruebas, 0 fallos, 0 errores y 0 omitidas.

7. Trazabilidad técnica

El registro de Android confirma que las respuestas fallidas no son inventadas por la interfaz: provienen de solicitudes Ktor al servicio REST. También se conservan los estados correctos de las operaciones exitosas.

```
● ● ● Logcat – solicitudes Ktor verificadas

09-29 21:10:37 Ktor REQUEST POST /api/v1/productos
09-29 21:10:38 Ktor RESPONSE 400 Bad Request

09-29 21:11:09 Ktor REQUEST DELETE /api/v1/productos/1
09-29 21:11:10 Ktor RESPONSE 409 Conflict

09-29 21:11:47 Ktor REQUEST DELETE /api/v1/productos/999
09-29 21:11:48 Ktor RESPONSE 404 Not Found

09-29 21:12:50 Ktor RESPONSE 201 Created
09-29 21:13:44 Ktor RESPONSE 200 OK
09-29 21:14:16 Ktor RESPONSE 204 No Content

Origen: adb logcat del emulador Pixel 6 API 37
```

Figura 12. Extracto legible de los eventos reales obtenidos con adb logcat.

```
● ● ● Prueba controlada – doble eliminación

> DELETE http://127.0.0.1:8080/api/v1/productos/2
HTTP/1.0 204 No Content

> DELETE http://127.0.0.1:8080/api/v1/productos/2
HTTP/1.0 404 Not Found
{ "message": "Producto no encontrado" }

Conclusión: el segundo intento se controla como NoEncontrado.
Ejecución real contra el mock REST local de la sesión.
```

Figura 13. Primera eliminación 204 y repetición 404 contra el mismo identificador.

8. Control de versiones y verificación final

Elemento	Resultado
Repositorio	https://github.com/DJuanParedes/pharmaMobil
Rama	actividad-autonoma-sesion-8
Commit funcional	19297b8 - test(sesion8): verificar estados y errores controlados
Commit de evidencias	f709146 - docs(sesion8): agregar evidencias de actividad autonoma
Pruebas	:shared:testAndroidHostTest - BUILD SUCCESSFUL; 54/54 aprobadas
APK	:androidApp:assembleDebug - BUILD SUCCESSFUL
Android	Pixel 6 API 37 - recorrido funcional y capturas reales
iOS	Código preparado; ejecución pendiente en macOS/Xcode

Conclusiones

1. La UI distingue validaciones locales, errores del servidor y problemas de red; por eso el usuario recibe una indicación concreta en lugar de un mensaje genérico.
2. Las operaciones que modifican productos pasan por EnCurso, regresan a Inactiva y recargan el listado, evitando datos desactualizados.
3. La cancelación mantiene su significado técnico: no se convierte en ErrorApi ni presenta un error inexistente al usuario.
4. La combinación de recorrido Android, Logcat y commonTest permite reproducir los resultados y relacionar cada captura con una respuesta concreta.

Pendiente honesto: para completar la evidencia visual multiplataforma se debe abrir el módulo iosApp en Xcode y repetir la matriz en un simulador iPhone. Esa acción no es técnicamente posible desde el equipo Windows usado para esta entrega.

Enlace de la rama de entrega: github.com/DJuanParedes/pharmaMobil/tree/actividad-autonoma-sesion-8